USB Device Activity Logger Driver

Project Report

Operating Systems [CS F372]

Semester II, 2025 – 2026

BITS Pilani
K K Birla Goa Campus

Course Instructor: Dr Biju K Raveendran

Submitted By

Group 36 – KernelX

Jaikumar Wath 2023A3PS0197G
Ayush Pareek 2023AAPS0609G

INDEX

1. Device Driver Implementation **3**
1.1 Project Setup 3
1.2 Header File Creation (usb_logger.h) 3
1.3 Driver File Creation (usb_logger.c) 4
1.4 Makefile Creation 7
1.5 Compilation of Kernel Module 7
1.6 Module Loading 8
1.7 Kernel Log Verification 8
2. User Space Implementation **9**
2.1 Test Application (test_app.c) 9
2.2 Compilation of User Application 10
2.3 Execution of User Application 10
3. Testing and Results **11**
3.1 USB Event Detection 11
3.2 Output via Character Device 12
3.3 Sysfs Interface Verification 12
3.4 Log Clearing Functionality 13
4. Conclusion **14**

1. Device Driver Implementation

1.1 Project Setup

A dedicated working directory was created to develop the USB Device Activity Logger driver. This directory contains all the files required for the kernel module, including the header file, driver source file, Makefile, and user-space application.

The following commands were used to create and navigate to the project directory:

mkdir kernelx_usb_logger

cd kernelx_usb_logger

This ensures that all project files are organized in a single location for easier compilation and testing.

1.2 Header File Creation (usb_logger.h)

A header file named `usb_logger.h` was created to define the data structures used by the driver.

The primary structure `usb_event` stores information related to each USB device event. It includes:

- Vendor ID
- Product ID
- Pointer to the USB device (used for accurate matching during removal)
- Device node (placeholder)
- Connection time
- Disconnection time
- Duration of connection
- Status of the device (ACTIVE or REMOVED)

Additionally, a circular buffer structure `usb_event_buffer` was implemented to store multiple USB events efficiently. It maintains:

- An array of events
- Head index for insertion
- Tail index for oldest event

- Count of events stored
- Spinlock for synchronization

The circular buffer ensures that when the buffer becomes full, older events are overwritten, allowing continuous logging without memory overflow.

```
GNU nano 7.2
#ifndef USB_LOGGER_H
#define USB_LOGGER_H

#include <linux/types.h>
#include <linux/usb.h>

#define MAX_EVENTS 100
#define EVENT_STATUS_ACTIVE 1
#define EVENT_STATUS_REMOVED 0

// Structure to hold USB event data
struct usb_event {
    uint16_t vendor_id;
    uint16_t product_id;
    const struct usb_device *device_ptr; // Bulletproof matching for disconnects
    char device_node[16];                // Placeholder for block layer assignment
    time64_t connect_time;
    time64_t disconnect_time;
    long duration_seconds;
    int status;                          // 1 = ACTIVE, 0 = REMOVED
};

// Circular buffer structure
struct usb_event_buffer {
    struct usb_event events[MAX_EVENTS];
    int head;
    int tail;
    int count;
    spinlock_t lock;
};

#endif // USB_LOGGER_H
```

Fig : usb_logger.h defining USB event structure and circular buffer

1.3 Driver File Creation (usb_logger.c)

The main functionality of the driver was implemented in the file `usb_logger.c`.

The driver registers with the USB subsystem using a notifier mechanism, which allows it to receive notifications when USB devices are inserted or removed.

Since notifier callbacks execute in atomic context, a workqueue mechanism is used to defer processing. This ensures that operations such as memory allocation and buffer updates are performed safely.

When a USB device is inserted:

- Vendor ID and Product ID are extracted
- Current timestamp is recorded

When a USB device is removed:

- The corresponding event is identified using pointer-based matching
- Disconnection time is recorded
- Duration is calculated
- Event status is updated to REMOVED

All events are stored in a circular buffer protected by a spinlock to ensure safe concurrent access.

```
GNU nano 7.2                                                    usb_logger.c
#include <linux/module.h>
#include <linux/init.h>
#include <linux/usb.h>
#include <linux/workqueue.h>
#include <linux/fs.h>
#include <linux/cdev.h>
#include <linux/uaccess.h>
#include <linux/ktime.h>
#include <linux/sysfs.h>
#include <linux/slab.h>
#include <linux/vmalloc.h>
#include <linux/rtc.h>
#include "usb_logger.h"

#define DRIVER_AUTHOR "Group 36 - KernelX"
#define DRIVER_DESC "USB Device Activity Logger"
#define DEVICE_NAME "usblogger"

// IST is UTC +5:30 (5 hours * 3600 + 30 mins * 60 = 19800 seconds)
#define IST_OFFSET_SECONDS 19800

static int major_number;
static struct class *logger_class;
static struct usb_event_buffer event_buf;
static struct workqueue_struct *usb_wq;

struct usb_work_data {
    struct work_struct work;
    struct usb_device *udev;
    unsigned long action;
};

// --- HELPER: FORMAT TIME (Now in IST) ---
static void format_time(time64_t ts, char *buf, size_t size) {
    struct rtc_time tm;

    // Add the IST offset to the raw UTC timestamp
    time64_t ts_ist = ts + IST_OFFSET_SECONDS;

    rtc_time64_to_tm(ts_ist, &tm);
    scnprintf(buf, size, "%02d:%02d:%02d", tm.tm_hour, tm.tm_min, tm.tm_sec);
}

// --- SYSFS INTERFACE ---
static ssize_t latest_event_show(struct kobject *kobj, struct kobj_attribute *attr, char *buf) {
    int idx;
    ssize_t len = 0;
```

```
GNU nano 7.2                                                    usb_logger.c
static ssize_t latest_event_show(struct kobject *kobj, struct kobj_attribute *attr, char *buf) {
    int idx;
    ssize_t len = 0;
    char conn_time_str[16];

    spin_lock(&event_buf.lock);
    if (event_buf.count == 0) {
        spin_unlock(&event_buf.lock);
        return scnprintf(buf, PAGE_SIZE, "No USB events logged yet.\n");
    }

    idx = (event_buf.head - 1 + MAX_EVENTS) % MAX_EVENTS;
    struct usb_event *e = &event_buf.events[idx];

    format_time(e->connect_time, conn_time_str, sizeof(conn_time_str));

    len += scnprintf(buf, PAGE_SIZE, "Vendor ID: %04x\nProduct ID: %04x\nConnection Time (IST): %s\nStatus: %s\n",
                    e->vendor_id, e->product_id, conn_time_str,
                    e->status == EVENT_STATUS_ACTIVE ? "ACTIVE" : "REMOVED");

    if (e->status == EVENT_STATUS_REMOVED) {
        char disc_time_str[16];
        format_time(e->disconnect_time, disc_time_str, sizeof(disc_time_str));
        len += scnprintf(buf + len, PAGE_SIZE - len, "Disconnection Time (IST): %s\nDuration: %ld seconds\n",
                        disc_time_str, e->duration_seconds);
    }

    spin_unlock(&event_buf.lock);
    return len;
}
static struct kobj_attribute latest_event_attr = __ATTR_RO(latest_event);
static struct kobject *logger_kobj;

// --- CHARACTER DEVICE OPERATIONS ---
static ssize_t logger_read(struct file *file, char __user *user_buffer, size_t size, loff_t *offset) {
    char *kbuf;
    struct usb_event *local_events;
    int len = 0, i, local_count;
    ssize_t bytes_to_copy;
    size_t alloc_size;

    if (*offset > 0) return 0;

    // 1. Safely grab the count
    spin_lock(&event_buf.lock);
    local_count = event_buf.count;
    spin_unlock(&event_buf.lock);
```

```
    spin_unlock(&event_buf.lock);

    if (local_count == 0) {
        char *empty_msg = "No USB activity logged.\n";
        bytes_to_copy = min((size_t)strlen(empty_msg), size);
        if (copy_to_user(user_buffer, empty_msg, bytes_to_copy)) return -EFAULT;
        *offset += bytes_to_copy;
        return bytes_to_copy;
    }

    alloc_size = max((size_t)256, (size_t)(local_count * 300));

    // 2. Allocate memory outside the lock
    kbuf = vzalloc(alloc_size);

    local_events = kmalloc_array(local_count, sizeof(struct usb_event), GFP_KERNEL);

    if (!kbuf || !local_events) {
        if (kbuf) vfree(kbuf);
        if (local_events) kfree(local_events);
        return -ENOMEM;
    }

    // 3. Snapshot the data FAST to minimize lock contention
    spin_lock(&event_buf.lock);
    for (i = 0; i < local_count; i++) {
        int idx = (event_buf.tail + i) % MAX_EVENTS;
        local_events[i] = event_buf.events[idx];
    }
    spin_unlock(&event_buf.lock);

    // 4. Format strings leisurely OUTSIDE the spinlock
    for (i = 0; i < local_count; i++) {
        char conn_time_str[16], disc_time_str[16];
        struct usb_event *e = &local_events[i];

        format_time(e->connect_time, conn_time_str, sizeof(conn_time_str));

        len += scnprintf(kbuf + len, alloc_size - len,
            "[EVENT %d]\nVendor ID: %04x\nProduct ID: %04x\nDevice Node: %s\nConnection Time (IST): %s\n",
            i + 1, e->vendor_id, e->product_id, e->device_node, conn_time_str);

        if (e->status == EVENT_STATUS_REMOVED) {
            format_time(e->disconnect_time, disc_time_str, sizeof(disc_time_str));
            long mins = e->duration_seconds / 60;
            long secs = e->duration_seconds % 60;
            len += scnprintf(kbuf + len, alloc_size - len,
                "Disconnection Time (IST): %s\nDuration: %ldm %lds\nStatus: REMOVED\n\n",
```

```
                "Disconnection Time (IST): %s\nDuration: %ldm %lds\nStatus: REMOVED\n\n",
                disc_time_str, mins, secs);
        } else {
            len += scnprintf(kbuf + len, alloc_size - len, "Status: ACTIVE\n\n");
        }
    }

    // 5. Send to user space
    bytes_to_copy = min((size_t)len, size);
    if (copy_to_user(user_buffer, kbuf, bytes_to_copy)) {
        vfree(kbuf);
        kfree(local_events);
        return -EFAULT;
    }

    *offset += bytes_to_copy;

    // 6. Cleanup
    vfree(kbuf);
    kfree(local_events);
    return bytes_to_copy;
}

static ssize_t logger_write(struct file *file, const char __user *user_buffer, size_t size, loff_t *offset) {
    spin_lock(&event_buf.lock);
    event_buf.head = 0;
    event_buf.tail = 0;
    event_buf.count = 0;
    spin_unlock(&event_buf.lock);
    printk(KERN_INFO "USB Logger: Logs cleared.\n");
    return size;
}

static struct file_operations logger_fops = {
    .owner = THIS_MODULE,
    .read = logger_read,
    .write = logger_write,
};

// --- WORKQUEUE HANDLER ---
static void usb_work_handler(struct work_struct *work) {
    struct usb_work_data *data = container_of(work, struct usb_work_data, work);
    struct usb_device *udev = data->udev;
    uint16_t vid = le16_to_cpu(udev->descriptor.idVendor);
    uint16_t pid = le16_to_cpu(udev->descriptor.idProduct);
    int i, idx;

```

```
GNU nano 7.2                                                    usb_l

    spin_lock(&event_buf.lock);

    if (data->action == USB_DEVICE_ADD) {
        struct usb_event *new_event = &event_buf.events[event_buf.head];
        new_event->vendor_id = vid;
        new_event->product_id = pid;
        new_event->device_ptr = udev; // Perfect matching pointer
        new_event->connect_time = ktime_get_real_seconds();
        new_event->status = EVENT_STATUS_ACTIVE;

        scnprintf(new_event->device_node, sizeof(new_event->device_node), "N/A (Block)");

        event_buf.head = (event_buf.head + 1) % MAX_EVENTS;
        if (event_buf.count < MAX_EVENTS) {
            event_buf.count++;
        } else {
            event_buf.tail = (event_buf.tail + 1) % MAX_EVENTS; // Overwrite oldest
        }
        printk(KERN_INFO "USB Logger: Inserted - VID: %04x, PID: %04x\n", vid, pid);

    } else if (data->action == USB_DEVICE_REMOVE) {
        for (i = 0; i < event_buf.count; i++) {
            idx = (event_buf.head - 1 - i + MAX_EVENTS) % MAX_EVENTS;
            // Match using the exact device pointer
            if (event_buf.events[idx].vendor_id == vid &&
                event_buf.events[idx].product_id == pid &&
                event_buf.events[idx].device_ptr == udev &&
                event_buf.events[idx].status == EVENT_STATUS_ACTIVE) {

                event_buf.events[idx].status = EVENT_STATUS_REMOVED;
                event_buf.events[idx].disconnect_time = ktime_get_real_seconds();
                event_buf.events[idx].duration_seconds =
                    event_buf.events[idx].disconnect_time - event_buf.events[idx].connect_time;
                break;
            }
        }
        printk(KERN_INFO "USB Logger: Removed - VID: %04x, PID: %04x\n", vid, pid);
    }

    spin_unlock(&event_buf.lock);
    kfree(data);
}

// --- USB NOTIFIER ---
static int usb_notify_callback(struct notifier_block *nb, unsigned long action, void *dev) {
    struct usb_work_data *work_data;
```

```
GNU nano 7.2                                                    u
// --- USB NOTIFIER ---
static int usb_notify_callback(struct notifier_block *nb, unsigned long action, void *dev) {
    struct usb_work_data *work_data;

    if (action != USB_DEVICE_ADD && action != USB_DEVICE_REMOVE) {
        return NOTIFY_DONE;
    }

    work_data = kmalloc(sizeof(*work_data), GFP_ATOMIC);
    if (!work_data) return NOTIFY_BAD;

    INIT_WORK(&work_data->work, usb_work_handler);
    work_data->udev = (struct usb_device *)dev;
    work_data->action = action;

    queue_work(usb_wq, &work_data->work);
    return NOTIFY_OK;
}

static struct notifier_block usb_nb = {
    .notifier_call = usb_notify_callback,
};

// --- MODULE INIT / EXIT ---
static int __init logger_init(void) {
    printk(KERN_INFO "USB Logger: Initializing...\n");

    spin_lock_init(&event_buf.lock);
    event_buf.head = 0;
    event_buf.tail = 0;
    event_buf.count = 0;

    usb_wq = alloc_workqueue("usb_logger_wq", WQ_UNBOUND, 1);
    usb_register_notify(&usb_nb);

    major_number = register_chrdev(0, DEVICE_NAME, &logger_fops);
    logger_class = class_create(THIS_MODULE, "logger_class");
    device_create(logger_class, NULL, MKDEV(major_number, 0), NULL, DEVICE_NAME);

    // FIRST create kobject
    logger_kobj = kobject_create_and_add("usblogger", kernel_kobj);
    if (!logger_kobj) {
        printk(KERN_ERR "USB Logger: Failed to create kobject\n");
        return -ENOMEM;
    }

    // THEN create sysfs file attached to it
```

```
    // THEN create sysfs file attached to it
    if (sysfs_create_file(logger_kobj, &latest_event_attr.attr)) {
        printk(KERN_ERR "USB Logger: Failed to create sysfs file\n");
    }

    return 0;
}

static void __exit logger_exit(void) {
    printk(KERN_INFO "USB Logger: Exiting...\n");
    usb_unregister_notify(&usb_nb);
    flush_workqueue(usb_wq);
    destroy_workqueue(usb_wq);

    kobject_put(logger_kobj);
    device_destroy(logger_class, MKDEV(major_number, 0));
    class_destroy(logger_class);
    unregister_chrdev(major_number, DEVICE_NAME);
}

module_init(logger_init);
module_exit(logger_exit);
MODULE_LICENSE("GPL");
MODULE_AUTHOR(DRIVER_AUTHOR);
MODULE_DESCRIPTION(DRIVER_DESC);
```

1.4 Makefile Creation

A Makefile was created to compile the driver as a loadable kernel module.

The Makefile specifies the module object and uses the kernel build system to compile the driver against the currently running kernel.

The module is compiled using:

make

This generates the kernel object file (`usb_logger.ko`), which can be loaded into the kernel.

```
  GNU nano 7.2
obj-m += usb_logger.o

KDIR := /lib/modules/$(shell uname -r)/build
PWD := $(shell pwd)

all:
        $(MAKE) -C $(KDIR) M=$(PWD) modules

clean:
        $(MAKE) -C $(KDIR) M=$(PWD) clean

load:
        sudo insmod usb_logger.ko
        sudo chmod 666 /dev/usblogger

unload:
        sudo rmmod usb_logger
```

Fig : Makefile used to compile and manage the kernel module

1.5 Compilation of Kernel Module

The driver was compiled using the kernel build system with the following command:

make

During compilation, the source code is processed and the kernel module file `usb_logger.ko` is generated.

The compilation completed successfully without critical errors, confirming that the driver is compatible with the kernel.

```
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ make
make -C /lib/modules/5.19.8/build M=/home/jai_wath_17/kernelx_usb_logger modules
make[1]: Entering directory '/usr/src/linux-5.19.8'
  CC [M]  /home/jai_wath_17/kernelx_usb_logger/usb_logger.o
/home/jai_wath_17/kernelx_usb_logger/usb_logger.c: In function 'latest_event_show':
/home/jai_wath_17/kernelx_usb_logger/usb_logger.c:57:5: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement]
   57 |     struct usb_event *e = &event_buf.events[idx];
      |     ^~~~~~
/home/jai_wath_17/kernelx_usb_logger/usb_logger.c: In function 'logger_read':
/home/jai_wath_17/kernelx_usb_logger/usb_logger.c:134:13: warning: ISO C90 forbids mixed declarations and code [-Wdeclaration-after-statement]
  134 |             long mins = e->duration_seconds / 60;
      |             ^~~~
  MODPOST /home/jai_wath_17/kernelx_usb_logger/Module.symvers
  CC [M]  /home/jai_wath_17/kernelx_usb_logger/usb_logger.mod.o
  LD [M]  /home/jai_wath_17/kernelx_usb_logger/usb_logger.ko
make[1]: Leaving directory '/usr/src/linux-5.19.8'
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ ls
Makefile  modules.order  Module.symvers  test_app  test_app.c  usb_logger.c  usb_logger.h  usb_logger.ko  usb_logger.mod  usb_logger.mod.c  usb_logger.mod.o  usb_logger.o
(base) jai_wath_17@Curse:~/kernelx_usb_logger$
```

Fig : Compilation output showing successful generation of usb_logger.ko

1.6 Module Loading

The compiled module was loaded into the kernel using:

make load

This command internally executes `insmod` to insert the module into the kernel and sets appropriate permissions for the device file.

```
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ make load
sudo insmod usb_logger.ko
sudo chmod 666 /dev/usblogger
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ lsmod | grep usb_logger
usb_logger             20480  0
(base) jai_wath_17@Curse:~/kernelx_usb_logger$
```

Fig : Terminal showing successful module loading and verification using lsmod

1.7 Kernel Log Verification

Kernel logs were checked to verify that the module initialized correctly:

sudo dmesg | grep "USB Logger"

The output shows initialization messages and logs generated during USB insertion and removal events. This confirms that the driver is functioning correctly and detecting USB activity.

2. User Space Implementation

2.1 Test Application (test_app.c)

A user-space application named `test_app.c` was developed to interact with the kernel module through the character device `/dev/usblogger`.

This application provides two main functionalities:

- Reading USB activity logs from the driver
- Clearing all stored logs in the kernel buffer

The program uses standard system calls such as `open()`, `read()`, and `write()` to communicate with the device file.

When the user runs the program with the `read` argument, it opens the device in read-only mode and retrieves formatted USB event logs.

When executed with the `clear` argument, it writes to the device file, which triggers the driver to reset the event buffer.

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

#define DEVICE_PATH "/dev/usblogger"

void read_logs() {
    int fd = open(DEVICE_PATH, O_RDONLY);
    if (fd < 0) {
        perror("Failed to open the device");
        return;
    }

    char buffer[8192]; // Large enough for multiple events
    ssize_t bytes_read = read(fd, buffer, sizeof(buffer) - 1);

    if (bytes_read >= 0) {
        buffer[bytes_read] = '\0';
        printf("--- USB Activity Logs ---\n%s\n", buffer);
    } else {
        perror("Failed to read from device");
    }
    close(fd);
}

void clear_logs() {
    int fd = open(DEVICE_PATH, O_WRONLY);
    if (fd < 0) {
        perror("Failed to open the device");
        return;
    }

    if (write(fd, "clear", 5) < 0) {
        perror("Failed to clear logs");
    } else {
        printf("Logs cleared successfully.\n");
    }
    close(fd);
}

int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s [read | clear]\n", argv[0]);
        return 1;
    }
```

```
int main(int argc, char *argv[]) {
    if (argc < 2) {
        printf("Usage: %s [read | clear]\n", argv[0]);
        return 1;
    }

    if (strcmp(argv[1], "read") == 0) {
        read_logs();
    } else if (strcmp(argv[1], "clear") == 0) {
        clear_logs();
    } else {
        printf("Invalid option.\n");
    }

    return 0;
}
```

2.2 Compilation of User Application

The user-space application was compiled using the GNU Compiler Collection (GCC) with the following command:

gcc test_app.c -o test_app

This generates an executable file named `test_app`.

```
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ gcc test_app.c -o test_app
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ ls
Makefile  modules.order  Module.symvers  test_app  test_app.c  usb_logger.c  usb_logger.h  usb_logger.ko  usb_logger.mod  usb_logger.mod.c  usb_logger.mod.o  usb_logger.o
(base) jai_wath_17@Curse:~/kernelx_usb_logger$
```

Fig : Compilation of test_app using GCC

2.3 Execution of User Application

The compiled application can be executed with different arguments to perform specific operations.

To read USB activity logs:

./test_app read

To clear all stored logs:

./test_app clear

The output of the `read` command displays detailed information about each USB event, including Vendor ID, Product ID, connection time, disconnection time, duration, and status.

```
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ ./test_app read
--- USB Activity Logs ---
[EVENT 1]
Vendor ID: 0781
Product ID: 5567
Device Node: N/A (Block)
Connection Time (IST): 20:48:43
Disconnection Time (IST): 20:48:59
Duration: 0m 16s
Status: REMOVED

[EVENT 2]
Vendor ID: 0781
Product ID: 55ab
Device Node: N/A (Block)
Connection Time (IST): 20:49:11
Disconnection Time (IST): 20:49:49
Duration: 0m 38s
Status: REMOVED

[EVENT 3]
Vendor ID: 0781
Product ID: 5567
Device Node: N/A (Block)
Connection Time (IST): 20:49:52
Status: ACTIVE


(base) jai_wath_17@Curse:~/kernelx_usb_logger$
```

Fig: Output of ./test_app read showing USB activity logs

3. Testing and Results

3.1 USB Event Detection

To verify the functionality of the driver, a USB device (pendrive) was inserted and removed.

The driver successfully detected both events using the USB notifier mechanism.

Kernel logs were checked using:

sudo dmesg | grep "USB Logger"

The output showed messages indicating USB insertion and removal along with the corresponding Vendor ID and Product ID.

This confirms that the driver correctly captures real-time USB events.

```
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ sudo dmesg | grep "USB Logger"
[  463.135316] USB Logger: Initializing...
[  495.674014] USB Logger: Removed - VID: 0781, PID: 5567
[  514.260294] USB Logger: Inserted - VID: 0781, PID: 5567
[  530.502573] USB Logger: Removed - VID: 0781, PID: 5567
[  542.981695] USB Logger: Inserted - VID: 0781, PID: 55ab
[  580.690445] USB Logger: Removed - VID: 0781, PID: 55ab
[  583.976810] USB Logger: Inserted - VID: 0781, PID: 5567
[  616.770538] USB Logger: Removed - VID: 0781, PID: 5567
(base) jai_wath_17@Curse:~/kernelx_usb_logger$
```

Fig : Kernel logs showing USB insertion and removal events

3.2 Output via Character Device

The user-space application was used to read the logged USB events from the kernel.

./test_app read

The output displayed detailed information about each USB event, including:

- Vendor ID
- Product ID
- Connection time
- Disconnection time
- Duration
- Status

This confirms that the driver correctly stores and exposes USB activity to user space.

3.3 Sysfs Interface Verification

The sysfs interface was used to access information about the most recent USB event.

cat /sys/kernel/usblogger/latest_event

The output displayed structured information including Vendor ID, Product ID, connection time, and status.

This verifies that the sysfs interface is correctly implemented.

```
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ cat /sys/kernel/usblogger/latest_event
Vendor ID: 0781
Product ID: 5567
Connection Time (IST): 20:49:52
Status: REMOVED
Disconnection Time (IST): 20:50:25
Duration: 33 seconds
(base) jai_wath_17@Curse:~/kernelx_usb_logger$
```

Fig : Sysfs output showing latest USB event

3.4 Log Clearing Functionality

The log clearing feature was tested using the user-space application.

./test_app clear

After clearing, logs were checked again:

./test_app read

The output confirmed that no USB activity was present, indicating that the buffer was successfully reset.

```
(base) jai_wath_17@Curse:~/kernelx_usb_logger$ ./test_app clear
./test_app read
Logs cleared successfully.
--- USB Activity Logs ---
No USB activity logged.

(base) jai_wath_17@Curse:~/kernelx_usb_logger$
```

Fig : Output showing logs cleared successfully

4. Conclusion

The USB Device Activity Logger successfully demonstrates the implementation of a Linux kernel module for monitoring USB device activity.

The driver efficiently detects USB insertion and removal events using the USB notifier mechanism and logs relevant information such as Vendor ID, Product ID, connection time, and duration. A circular buffer is used to store events, ensuring efficient memory usage and continuous logging.

The logged data is made accessible to user space through a character device interface and a sysfs entry, enabling easy interaction and verification.

This project provides practical exposure to important kernel programming concepts such as workqueues, synchronization using spinlocks, kernel-user space communication, and device driver development.